

Lista Avaliativa – Estrutura de Dados: Funções, Ponteiros e Recursividade
Professora: Dra. Maíla de Lima Claro Disciplina: Estrutura de Dados I Tema:
Funções, Passagem por Valor e Referência, Ponteiros e Recursividade

Discente: Felipe Pereira Coelho

1. Dobro real

Escreva uma função void dobraValor(int *n) que dobre o valor da variável passada como argumento. No main, peça ao usuário um número, chame a função e exiba o valor original e o dobrado.

```
void dobraValor(int *n) {  
    // O operador '*' acessa o valor no endereço de memória  
    // apontado por 'n'. O valor é multiplicado por 2 e o resultado é  
    // armazenado de volta no mesmo endereço, modificando a variável original.  
    *n = *n * 2;  
}  
  
int main() {  
    int num; // Variável para armazenar o número digitado.  
    int *pont; // Ponteiro para armazenar o endereço de 'num'.  
    printf("Digite um numero: ");  
    scanf("%d", &num);  
    // 'pont' recebe o endereço de memória da variável 'num'.  
    pont = &num;  
    // Exibe o valor original antes de chamar a função.  
    printf("O dobro de %d", num);  
    // Chamada da função 'dobraValor', passando o endereço (ponteiro) de 'num'.  
    // Isso permite que a função altere o valor de 'num' na main.  
    dobraValor(pont);  
    // Exibe o valor de 'num' após a modificação pela função (o valor dobrado).  
    printf(" eh %d\n", num);  
    return 0;  
}
```

```
}
```

Print do código em execução:

```
• Digite um numero: 5
  O dobro de 5 eh 10
```

2. Troca de valores

Implemente uma função void troca(int *a, int *b) que troque os valores de duas variáveis. Teste com x = 10 e y = 20, imprimindo o resultado final.

//Função troca inverte os valores das duas variáveis inteiras apontadas pelos ponteiros.

//Parâmetros:

//int *a - Ponteiro para a primeira variável (referência para x).

//int *b - Ponteiro para a segunda variável (referência para y).

//Retorno: void - Não retorna valor, pois modifica os dados diretamente nos endereços.

```
void troca (int *a, int *b) {
```

```
    int aux; // Variável auxiliar necessária para armazenar temporariamente um valor.
```

```
    // Armazena o valor apontado por 'a' (*a) na variável auxiliar.
```

```
    // (Ex: Se 'a' aponta para x=10, aux = 10)
```

```
    aux = *a;
```

```
    // O valor apontado por 'b' (*b) é copiado para o endereço de 'a' (*a).
```

```
    // (Ex: x recebe o valor de y. x agora é 20)
```

```
    *a = *b;
```

```
    // 3. O valor armazenado em 'aux' (o valor original de 'a') é copiado para o endereço de 'b' (*b).
```

```
    // (Ex: y recebe o valor de aux. y agora é 10)
```

```
    *b = aux;
```

```
}
```

```
int main() {
```

```
    // Variáveis originais com valores iniciais.
```

```
    int x = 10, y = 20;
```

```
    // Ponteiros para armazenar os endereços de 'x' e 'y'.
```

```

int *px, *py;

// 'px' recebe o endereço de 'x', e 'py' recebe o endereço de 'y'.

px = &x;

py = &y;

// Exibe os valores ANTES da troca.

printf("x = %d, y = %d\n", x, y); // Output: x = 10, y = 20

// Chama a função 'troca', passando os endereços (ponteiros) das variáveis.

// Isso permite a modificação das variáveis originais.

troca(px, py);

// Exibe os valores DEPOIS da troca.

printf("x = %d, y = %d\n", x, y);

return 0;

}

```

Print do código em execução:

```

x = 10, y = 20
x = 20, y = 10

```

3. Maior e menor

Crie uma função void maiorMenor(int *v, int n, int *maior, int *menor) que encontre o maior e o menor número de um vetor de inteiros. Use um vetor com 10 números digitados pelo usuário.

```

/*
* Função: maiorMenor
* Objetivo: Encontra o maior e o menor valor em um vetor de inteiros.
* * Parâmetros:
* int *v - Ponteiro para o primeiro elemento do vetor.
* int n - O número de elementos no vetor.
* int *maior - Ponteiro para a variável onde o resultado MAIOR será armazenado.
* int *menor - Ponteiro para a variável onde o resultado MENOR será armazenado.
* * Retorno: void - Não retorna valor, pois usa ponteiros para modificar 'maior' e 'menor'
na main.

```

```
*/
```

```
void maiorMenor(int *v, int n, int *maior, int *menor) {
```

```
    // Inicializa as variáveis de resultado (nas posições de memória de 'maior' e 'menor' na main)
```

```
    // com o valor do primeiro elemento do vetor.
```

```
    *maior = v[0];
```

```
    *menor = v[0];
```

```
    // Itera sobre o restante do vetor, começando pelo segundo elemento (i = 1).
```

```
    for (int i = 1; i < n; i++) {
```

```
        // Compara o elemento atual v[i] com o maior valor encontrado até agora (*maior).
```

```
        if (v[i] > *maior)
```

```
            // Se for maior, *maior (o conteúdo do endereço) é atualizado.
```

```
            *maior = v[i];
```

```
        // Compara o elemento atual v[i] com o menor valor encontrado até agora (*menor).
```

```
        if (v[i] < *menor)
```

```
            // Se for menor, *menor (o conteúdo do endereço) é atualizado.
```

```
            *menor = v[i];
```

```
    }
```

```
}
```

```
int main() {
```

```
    // Declara um vetor de 10 posições e as variáveis para os resultados.
```

```
    int vetor[10], maior, menor;
```

```
    printf("Digite 10 numeros inteiros:\n");
```

```
    // Loop para ler os 10 números e preencher o vetor.
```

```
    for (int i = 0; i < 10; i++) {
```

```
        printf("Numero %d: ", i + 1);
```

```
        // O operador '&' é crucial para passar o endereço do elemento do vetor para scanf.
```

```
        scanf("%d", &vetor[i]);
```

```
    }
```

```

// Chama a função, passando o vetor, seu tamanho (10) e os ENDEREÇOS (&)
// das variáveis 'maior' e 'menor' para que a função possa modificá-las.

maiorMenor(vetor, 10, &maior, &menor);

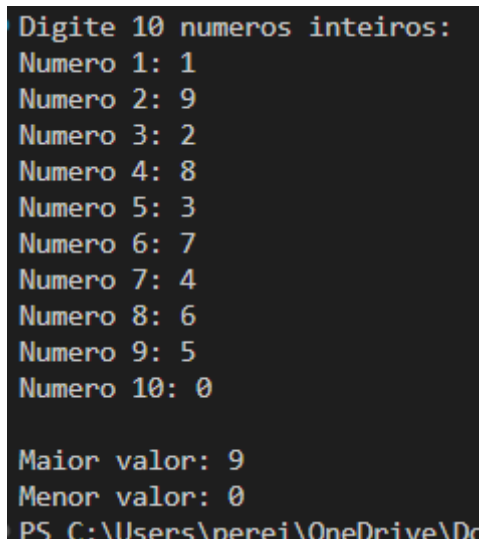
// Exibe os resultados que foram armazenados nas variáveis locais 'maior' e 'menor'
// pela função maiorMenor.

printf("\nMaior valor: %d\n", maior);
printf("Menor valor: %d\n", menor);

return 0;
}

```

Print do código em execução:



```

Digite 10 numeros inteiros:
Numero 1: 1
Numero 2: 9
Numero 3: 2
Numero 4: 8
Numero 5: 3
Numero 6: 7
Numero 7: 4
Numero 8: 6
Numero 9: 5
Numero 10: 0

Maior valor: 9
Menor valor: 0
PS C:\Users\pereir\OneDrive\De

```

4. Contar pares recursivamente

Implemente uma função recursiva `int contaPares(int *v, int n)` que retorne quantos números pares existem em um vetor.

```
/**Função: contarPares
```

* Objetivo: Conta recursivamente quantos números pares existem em um vetor.

* Parâmetros:

* `int *v`: Ponteiro para o vetor (ou sub-vetor).

* `int n`: Tamanho restante do vetor a ser analisado.

* Retorno: int - O número total de pares encontrados.

*/

```
int contarPares(int *v, int n) {
```

```
    // Caso Base: Se o vetor estiver vazio, retorna 0.
```

```
    if (n <= 0) {
```

```
        return 0;
```

```
    }
```

```
    // Passo Recursivo:
```

```
    // Verifica se o primeiro elemento atual é par.
```

```
    if (v[0] % 2 == 0) {
```

```
        // Se for par, retorna 1 (para este elemento) + a contagem do restante do vetor.
```

```
        // v + 1 avança o ponteiro; n - 1 diminui o tamanho.
```

```
        return 1 + contarPares(v + 1, n - 1);
```

```
    } else {
```

```
        // Se for ímpar, retorna 0 (implicitamente) + a contagem do restante do vetor.
```

```
        return contarPares(v + 1, n - 1);
```

```
    }
```

```
}
```

```
int main() {
```

```
    // Vetor de teste: {1, 2, 3, 4}. Pares são 2 e 4.
```

```
    int vetor[] = {1, 2, 3, 4};
```

```
    int tamanho = 4;
```

```
    int num_pares = contarPares(vetor, tamanho);
```

```
    printf("No vetor {1, 2, 3, 4}, o numero de pares eh: %d\n", num_pares);
```

```
    return 0;
```

```
}
```

Print do código em execução:

```
No vetor {1, 2, 3, 4}, o numero de pares eh: 2
```

5. Reodernar

Faça um programa em C que leia três números inteiros e os reordene de forma que o maior fique na primeira variável, o médio na segunda e o menor na terceira. A reorganização deve ser feita dentro de uma função chamada ordenarTres(), utilizando apenas ponteiros (sem usar vetores, laços ou bibliotecas de ordenação).

```
/*
* Função: ordenarTres
* Objetivo: Ordena os três valores passados por referência em ordem decrescente (Maior
-> Menor).
* Parâmetros: int *valor1, int *valor2, int *valor3 - Ponteiros para as variáveis a serem
modificadas.
* Retorno: void - Modifica as variáveis diretamente via ponteiros.
*/
void ordenarTres(int *valor1, int *valor2, int *valor3) {
    int aux = 0; // Variável auxiliar para realizar as trocas (swap).

    // Se o *valor2 é o maior de todos.
    if (*valor2 >= *valor1 && *valor2 >= *valor3) {
        // Traz o maior valor (*valor2) para a primeira posição (*valor1).
        aux = *valor1; // Salva o valor original de *valor1.
        *valor1 = *valor2; // *valor1 recebe o maior valor (*valor2).
        // Agora, *valor1 (maior) está correto. Precisamos ordenar o valor original de *valor1
        (em aux)
        // e o valor de *valor3 entre as posições *valor2 e *valor3.
        if (aux <= *valor3) {
            // Se o original de *valor1 (aux) for o menor (menor ou igual a *valor3),
            // a ordem correta é *valor3 (interm.) em *valor2, e aux (menor) em *valor3.
            *valor2 = *valor3;
            *valor3 = aux;
        } else {
            // Se o original de *valor1 (aux) for o interm., a ordem correta é:
            // aux (interm.) em *valor2, e *valor3 (menor) já está na posição correta.
            *valor2 = aux;
```

```

    }
}

// Se o *valor3 é o maior de todos.
else if (*valor3 >= *valor1 && *valor3 >= *valor2) {
    // Traz o maior valor (*valor3) para a primeira posição (*valor1).
    aux = *valor1; // Salva o valor original de *valor1.
    *valor1 = *valor3; // *valor1 recebe o maior valor (*valor3).
    // Agora, *valor1 (maior) está correto. Ordena o valor original de *valor1 (aux)
    // e o valor de *valor2 entre as posições *valor2 e *valor3.
    if (aux >= *valor2) {
        // Se o original de *valor1 (aux) for o interm. ou menor (interm. ou maior que
        *valor2),
        // a ordem correta é aux (interm.) em *valor2, e *valor2 (menor) em *valor3.
        *valor3 = *valor2;
        *valor2 = aux;
    } else {
        // Se o original de *valor1 (aux) for o menor (menor que *valor2),
        // a ordem correta é *valor2 (interm.) em *valor2, e aux (menor) em *valor3.
        *valor3 = aux;
    }
}

// *valor1 já é o maior de todos.
else {
    // Apenas precisamos garantir que *valor2 seja maior ou igual a *valor3.
    if (*valor3 >= *valor2) {
        // Se *valor3 é maior, troca *valor2 e *valor3.
        aux = *valor2;
        *valor2 = *valor3;
        *valor3 = aux;
    }

    // Se *valor2 > *valor3, a ordem já está correta.

```



```

    }
}

int main() {

    // Valores para teste.

    int a = 2, b = 3, c = 1;

    // Declaração de ponteiros e atribuição de endereços.

    int *pa = &a, *pb = &b, *pc = &c;


    printf("Valores ANTES da ordenacao:\n");
    printf("a = %d\nb = %d\nc = %d\n", a, b, c);

    // Chamada da função, passando os endereços (ponteiros) das variáveis.

    ordenarTres(pa, pb, pc);


    printf("\nValores DEPOIS da ordenacao (Decrescente):\n");

    // Exibe os valores finais

    printf("a = %d\nb = %d\nc = %d", a, b, c);


    return 0;

}

```

Prints do código em execução:

```

Valores ANTES da ordenacao:
a = 2
b = 3
c = 1

Valores DEPOIS da ordenacao (Decrescente):
a = 3
b = 2
c = 1

```

6. Fibonacci com ponteiro

Crie uma função void fibonacci(int n, int *resultado) que, de forma recursiva, calcule o n-ésimo termo da série de Fibonacci e armazene o valor em *resultado.

```
/*  
* Função: fibonacci  
* Objetivo: Calcula recursivamente o n-ésimo termo da Sequência de Fibonacci.  
* Parâmetros:  
* int n - O índice do termo da sequência a ser calculado (ex: n=6 para F6).  
* int *resultado - Ponteiro para a variável onde o valor calculado será armazenado.  
* Retorno: void - Não retorna valor, pois usa o ponteiro para modificar o dado.  
*/  
  
void fibonacci(int n, int *resultado) {  
    // Primeiro Caso Base: F0 = 0 e para n negativo.  
    if (n <= 0) {  
        *resultado = 0;  
    }  
    // Segundo Caso Base: F1 = 1.  
    else if (n == 1) {  
        *resultado = 1;  
    }  
    // Passo Recursivo: Fn = F(n-1) + F(n-2).  
    else {  
        int valor_n_1, valor_n_2; // Variáveis locais para armazenar os resultados das sub-  
        chamadas.  
        // Chamada recursiva para calcular F(n-1).  
        // Passamos o endereço de &valor_n_1 para armazenar o resultado aqui.  
        fibonacci(n - 1, &valor_n_1);  
        // Chamada recursiva para calcular F(n-2).  
        // Passamos o endereço de &valor_n_2 para armazenar o resultado aqui.  
        fibonacci(n - 2, &valor_n_2);  
        // O resultado final é a soma dos dois termos anteriores, armazenado
```

```

    // no endereço de memória original (passado por *resultado).

    *resultado = valor_n_1 + valor_n_2;
}
}

int main() {
    int resultado = 0, n = 0; // Variável que irá receber o resultado final.
    int *presult; // Ponteiro para referenciar a variável 'resultado'.

    presult = &resultado; // 'presult' aponta para o endereço de 'resultado'.

    printf("Qual termo da sequencia de fibonacci deseja saber? ");
    scanf("%d", &n);

    // Chama a função para calcular o n termo de Fibonacci.

    // O valor será escrito no endereço apontado por 'presult' (ou seja, na variável
    'resultado').

    fibonacci(n, presult);

    // Exibe o valor de 'resultado' após a execução da função.

    printf("O %d termo da sequencia eh: %d\n", n, resultado);

    return 0;
}

```

Prints do código em execução:

```

Qual termo da sequencia de fibonacci deseja saber? 5
0 5 termo da sequencia eh: 5

```

```

Qual termo da sequencia de fibonacci deseja saber? 3
0 3 termo da sequencia eh: 2

```

```

Qual termo da sequencia de fibonacci deseja saber? 7
0 7 termo da sequencia eh: 13

```

7. Estatísticas do vetor

Faça um programa que leia N números inteiros e utilize funções para: calcular a soma, a média e os extremos (maior e menor valor). Use passagem por referência nos parâmetros.

```
/** Função: soma_valores
```

```
* Objetivo: Calcula a soma dos elementos de um vetor.
```

```
* Parâmetros: int v[] (vetor), int n (tamanho), int *soma (saída por referência). */
```

```
void soma_valores(int v[], int n, int *soma) {
```

```
    // Itera sobre o vetor e acumula a soma no endereço de memória do ponteiro *soma.
```

```
    // Presume-se que *soma foi inicializado com 0 antes da chamada.
```

```
    for (int i = 0 ; i < n ; i++) {
```

```
        *soma += v[i];
```

```
    }
```

```
}
```

```
/** Função: media_valores
```

```
* Objetivo: Calcula a média dos elementos de um vetor.
```

```
* Parâmetros: int v[] (vetor), int n (tamanho), float *media (saída por referência).*/
```

```
void media_valores(int v[], int n, float *media) {
```

```
    float soma = 0; // Soma local, deve ser float para precisão.
```

```
    // Acumula a soma dos elementos.
```

```
    for (int i = 0 ; i < n ; i++) {
```

```
        soma += v[i];
```

```
    }
```

```
    // Calcula a média e armazena o resultado no endereço de memória de *media.
```

```
    *media = soma / n;
```

```
}
```

```
/** Função: extremos_valores
```

```
* Objetivo: Encontra o maior e o menor valor do vetor.
```

* Parâmetros: int v[] (vetor), int n (tamanho), int *maior, int *menor (saída por referência). */

```
void extremos_valores(int v[], int n, int *maior, int *menor) {  
    // *maior e *menor deveriam ser inicializados com v[0].  
    // Inicialização do maior (usando 0, falha se todos forem negativos)  
    *maior = v[0];  
    // Inicialização do menor (usando o primeiro elemento)  
    *menor = v[0];  
    // Varre o vetor para encontrar os extremos.  
    for ( int i = 0 ; i < n ; i++) {  
        // Se o elemento atual for maior ou igual ao maior atual, atualiza *maior.  
        if (v[i] >= *maior) {  
            *maior = v[i];  
        }  
        // Se o elemento atual for menor ou igual ao menor atual, atualiza *menor.  
        if (v[i] <= *menor) {  
            *menor = v[i];  
        }  
    }  
}  
  
int main() {  
    int n;  
    printf("Quantos valores deseja digitar? ");  
    scanf("%d", &n);  
    int valores[n];  
    for (int i = 0 ; i < n ; i++) {  
        printf("Digite o numero %d: ", i + 1);  
        scanf("%d", &valores[i]);  
    }  
    int soma = 0, maior = 0, menor = 0;  
    float media = 0;
```

```

// Chamadas das funções, passando os endereços (&) das variáveis de resultado.

soma_valores(valores, n, &soma);
media_valores(valores, n, &media);
extremos_valores(valores, n, &maior, &menor);

// Exibição dos resultados finais obtidos através da modificação por referência.

printf("\n--- Resultados ---\n");
printf("Soma: %d\n", soma);
printf("Media: %.1f\n", media);
printf("Maior: %d\n", maior);
printf("Menor: %d\n", menor);

return 0;
}

```

Prints do código em execução:

```

Quantos valores deseja digitar? 5
Digite o numero 1: 50
Digite o numero 2: 30
Digite o numero 3: 10
Digite o numero 4: 20
Digite o numero 5: 40

--- Resultados ---
Soma: 150
Media: 30.0
Maior: 50
Menor: 10

```

```

Quantos valores deseja digitar? 8
Digite o numero 1: 100
Digite o numero 2: 130
Digite o numero 3: 20
Digite o numero 4: 48
Digite o numero 5: 10
Digite o numero 6: 135
Digite o numero 7: 200
Digite o numero 8: 197

--- Resultados ---
Soma: 840
Media: 105.0
Maior: 200
Menor: 10

```

8. Ordenação com ponteiros

Implemente o algoritmo Bubble Sort usando ponteiros em vez de índices.

Protótipo: void bubbleSort(int *v, int n).

```
/** Função: bubbleSort
```

```
* Objetivo: Ordena um vetor de inteiros em ordem crescente usando o algoritmo Bubble Sort.
```

```
* O acesso e a troca de elementos são realizados exclusivamente através de ponteiros.
```

```
* Parâmetros:
```

```
* int *v - Ponteiro para o primeiro elemento do vetor.
```

```
* int n - O número de elementos no vetor.
```

```
* Retorno: void - Modifica o vetor diretamente na memória. */
```

```
void bubbleSort(int *v, int n) {
```

```
    // Loop externo: Controla o número de passagens necessárias.
```

```
    for (int i = 0; i < n - 1; i++) {
```

```
        // Loop interno: Realiza as comparações e trocas na passagem atual.
```

```
        for (int j = 0; j < n - 1 - i; j++) {
```

```
            // Aritmética de ponteiros: 'a' aponta para o elemento atual (v[j]).
```

```
            int *a = v + j;
```

```
            // Aritmética de ponteiros: 'b' aponta para o elemento adjacente (v[j+1]).
```

```
            int *b = v + j + 1;
```

```
            // Comparação dos VALORES nos endereços apontados por 'a' e 'b'.
```

```
            if (*a > *b) {
```

```
                // Realiza a troca.
```

```
                int temp = *a; // 1. Salva o valor de *a.
```

```
                *a = *b;    // 2. *a recebe o valor de *b.
```

```
                *b = temp;  // 3. *b recebe o valor original de *a.
```

```
            }
```

```
        }
```

```
    }
```

```
}
```

```
int main() {
```

```

// Inicialização do vetor e do ponteiro para o início do vetor.

int vetor[3] = {2,3,1};

int *p = vetor;

// Cálculo do tamanho do vetor.

int tamanho = sizeof(vetor) / sizeof(vetor[0]);

printf("Vetor ANTES da ordenacao: \n");

for ( int i = 0 ; i < tamanho ; i++ ) {

    printf("[%d]", vetor[i]);

}

// Chama a função de ordenação passando o ponteiro (p) e o tamanho.

bubbleSort(p, tamanho);

printf("\nVetor DEPOIS da ordenacao: \n");

// Exibe o vetor ordenado.

for ( int i = 0 ; i < tamanho ; i++ ) {

    printf("[%d]", vetor[i]);

}

printf("\n");

return 0;

}

```

Prints do código em execução:

```

Vetor ANTES da ordenacao:
[2][3][1]
Vetor DEPOIS da ordenacao:
[1][2][3]

```

```

Vetor ANTES da ordenacao:
[5][2][3][7][4][1]
Vetor DEPOIS da ordenacao:
[1][2][3][4][5][7]

```


9. Soma de dígitos recursiva

Crie uma função `int somaDigitos(int n)` que retorne a soma dos dígitos de um número.

Exemplo: `somaDigitos(1234) → 10`.

```
/** Função: somaDigitos
```

```
* Objetivo: Calcula a soma dos dígitos de um número inteiro positivo de forma recursiva.
```

```
* Parâmetro: int i - O número inteiro cuja soma dos dígitos será calculada.
```

```
* Retorno: int - A soma total dos dígitos. */
```

```
int somaDigitos(int i) {
```

```
    // 1. Caso Base: Condição de parada da recursão.
```

```
    // Se 'i' for um único dígito, ele próprio é o último a ser somado.
```

```
    if ( i < 10 ) {
```

```
        return i;
```

```
    }
```

```
    // 2. Passo Recursivo: Decomposição do número.
```

```
    else {
```

```
        // i % 10: Isola o último dígito (unidade).
```

```
        // i / 10: Remove o último dígito e chama a função recursivamente com o resto do número.
```

```
        return i % 10 + somaDigitos(i / 10);
```

```
    }
```

```
}
```

```
int main() {
```

```
    int valor;
```

```
    printf("Digite um valor: ");
```

```
    scanf("%d", &valor);
```

```
    // Chama a função recursiva e exibe o resultado.
```

```
    printf("A soma dos digitos de %d eh %d\n", valor ,somaDigitos(valor));
```

```
    return 0;
```

```
}
```

Prints do código em execução:

```
Digite um valor: 1234
A soma dos digitos de 1234 eh 10
```

```
Digite um valor: 5647
A soma dos digitos de 5647 eh 22
```

```
Digite um valor: 1001
A soma dos digitos de 1001 eh 2
```

10. Faça um programa em C que dado uma matriz de inteiros, calcule o produto entre a matriz original e sua matriz transposta. O programa deve implementar a função que monte a matriz transposta e a função que calcula o produto, além das funções para ler e imprimir uma matriz.

Obs. 1: Deve ser permitido ao usuário definir a ordem da matriz original, desde que não ultrapasse o máximo definido no programa.

Obs. 2: As funções imprime, transposta e produto devem ser recursivas.

Obs. 3: Deve existir uma única função imprime, deve ser possível imprimir qualquer matriz de inteiros utilizando esta função.

```
/**Função: imprimeMatriz
```

```
* Objetivo: Imprime o conteúdo de uma matriz de forma recursiva.
```

```
* Parâmetros:
```

```
* int lin, int col: Dimensões da matriz.
```

```
* int matriz[lin][col]: A matriz a ser impressa (VLA).
```

```
* int c: Contador da Linha Atual (c).
```

```
* int l: Contador da Coluna Atual (l).
```

```
*/
```

```
void imprimeMatriz(int lin, int col, int matriz[lin][col], int c, int l) {
```

```
    // Caso Base 1: Fim das linhas (c == col, pois 'c' itera sobre as linhas da matriz que está na 1ª dimensão)
```

```
    if (c == lin) { // Usando 'lin' como o limite da primeira dimensão
```

```
        return; // return vazio pois se trata de uma função do tipo 'void'
```

```
}
```

```

// Caso Base 2: Fim da coluna atual
if (l == col) {
    printf("\n"); // Quebra de linha
    // Próxima linha, coluna reinicia em 0
    imprimeMatriz(lin, col, matriz, c + 1, 0);
    return;
}

// Passo Recursivo: Imprime o elemento e avança para a próxima coluna (l + 1)
printf("[%d]", matriz[c][l]);
imprimeMatriz(lin, col, matriz, c, l + 1);
}

/**Função: matrizTransposta
* Objetivo: Calcula a matriz transposta ( $A^T$ ) de forma recursiva.
* Parâmetros:
* int lin, int col: Dimensões da matriz original A.
* int matriz[lin][col]: Matriz original A.
* int matrizT[col][lin]: Matriz transposta  $A^T$  (saída).
* int i: Contador da linha de A.
* int j: Contador da coluna de A.
*/
void matrizTransposta(int lin, int col, int matriz[lin][col], int matrizT[col][lin], int i, int j) {
    // Caso Base 1: Fim das linhas da matriz original (i == lin)
    if (lin == i) {
        return;
    }
    // Caso Base 2: Fim das colunas da linha atual (j == col)
    if (col == j) {
        // Move para a próxima linha (i + 1) e reinicia a coluna (j = 0)
        matrizTransposta(lin, col, matriz, matrizT, i + 1, 0);
    }
}

```

```

        return;
    }

    // Passo Recursivo: Executa a transposição  $A^T[j][i] = A[i][j]$ 
    matrizT[j][i] = matriz[i][j];

    // Avança para a próxima coluna (j + 1)
    matrizTransposta(lin, col, matriz, matrizT, i, j + 1);
}

/**Função: matrizResultante

* Objetivo: Calcula o produto de matrizes de forma recursiva ( $C[i][j] = A[i][k] * B[k][j]$ ).
* Parâmetros:
* int linA, int colA, int colB: Dimensões da operação (A é linA x colA, B é colA x colB).
* int matriz[linA][colA]: Matriz A.
* int matrizT[colA][colB]: Matriz B (neste caso,  $A^T$ ).
* int matrizResult[linA][colB]: Matriz Resultado (saída).
* int i: Linha atual da matriz resultante.
* int j: Coluna atual da matriz resultante.
* int k: Índice do elemento da soma interna.
*/

void matrizResultante(int linA, int colA, int colB, int matriz[linA][colA], int
matrizT[colA][colB], int matrizResult[linA][colB], int i, int j, int k) {

    // Fim das linhas da matriz resultante (i == linA)
    if (linA == i) {
        return;
    }

    // Fim da coluna 'j' da matriz resultante (j == colB)
    if (colB == j) {
        // Move para a próxima linha (i + 1) e reinicia as colunas e a soma interna (j=0, k=0)
        matrizResultante(linA, colA, colB, matriz, matrizT, matrizResult, i + 1, 0, 0);
        return;
    }

```

```

// Fim da soma interna (k == colA)

else if (colA == k) {

    // A soma para a posição [i][j] está completa. Move para a próxima coluna (j + 1) e
    reinicia a soma (k=0)

    matrizResultante(linA, colA, colB, matriz, matrizT, matrizResult, i, j + 1, 0);

    return;

}

// Passo Recursivo: Executa a multiplicação e acumulação

else {

    // MatrizResult[i][j] += MatrizA[i][k] * MatrizT[k][j]

    matrizResult[i][j] += matriz[i][k] * matrizT[k][j];

    // Move para o próximo elemento da soma interna (k + 1)

    matrizResultante(linA, colA, colB, matriz, matrizT, matrizResult, i, j, k + 1);

}

}

int main() {

    int lin = 0, col = 0;

    printf("Informe a ordem da matriz (linha coluna): ");

    scanf("%d %d", &lin, &col);

    int matriz[lin][col];

    int matrizT[col][lin];

    // Matriz resultante A x A^T terá dimensões (lin (linha da primeira) x lin (coluna da
    segunda)

    int resultMatriz[lin][lin];

    // Inicializa a matriz resultante com zeros (crucial para a soma na multiplicação).

    for (int i = 0; i < lin; i++) {

        for (int j = 0; j < lin; j++) {

            resultMatriz[i][j] = 0;

        }

    }

    // Coleta dos valores da matriz A.

    for (int i = 0; i < lin ; i++) { // Laço por linhas

```

```

for ( int j = 0 ; j < col ; j++ ) { // Laço por colunas

    printf("Valor da posicao [%d][%d]: ", i, j);

    int valor = 0;

    scanf("%d", &valor);

    matriz[i][j] = valor;

}

}

printf("\nMatriz Original (%d x %d)\n", lin, col);

imprimeMatriz(lin, col, matriz, 0, 0);

// 1. Transpõe a matriz

matrizTransposta(lin, col, matriz, matrizT, 0, 0);

printf("\nMatriz Transposta (%d x %d)\n", col, lin);

// Nota: Passei as dimensões invertidas para imprimir a matrizT corretamente.

imprimeMatriz(col, lin, matrizT, 0, 0);

// 2. Multiplica a matriz original pela transposta:  $A(lin \times col) * A^T(col \times lin) = C(lin \times lin)$ 

matrizResultante(lin, col, lin, matriz, matrizT, resultMatriz, 0, 0, 0);

printf("\nProduto das Duas Matrices (A x A^T) (%d x %d)\n", lin, lin);

imprimeMatriz(lin, lin, resultMatriz, 0, 0);

return 0;

}

```

Prints do código em execução:

```

Informe a ordem da matriz (linha coluna): 2 2
Valor da posicao [0][0]: 1
Valor da posicao [0][1]: 2
Valor da posicao [1][0]: 3
Valor da posicao [1][1]: 4

Matriz Original (2 x 2)
[1][2]
[3][4]

Matriz Transposta (2 x 2)
[1][3]
[2][4]

Produto das Duas Matrices (A x A^T) (2 x 2)
[5][11]
[11][25]

```

Informe a ordem da matriz (linha coluna): 2 3

Valor da posicao [0][0]: 1

Valor da posicao [0][1]: 2

Valor da posicao [0][2]: 3

Valor da posicao [1][0]: 4

Valor da posicao [1][1]: 5

Valor da posicao [1][2]: 6

Matriz Original (2 x 3)

[1][2][3]

[4][5][6]

Matriz Transposta (3 x 2)

[1][4]

[2][5]

[3][6]

Produto das Duas Matrizes ($A \times A^T$) (2 x 2)

[14][32]

[32][77]